

PRÁCTICA 3

Sincronización de procesos con mutex: Uso de mutex para resolver carreras críticas

ÍNDICE

Introducción	1
Códigos	1
Conclusiones	3

INTRODUCCIÓN

Para el apartado 2 de la práctica, se pide utilizar *mutex* y variables de condición para eliminar las condiciones de carrera del código del apartado 1.

Se ubicarán *sleeps* en regiones del código que permitan presenciar diferentes velocidades de ejecución tanto del productor como del consumidor.

CÓDIGOS

Se trabajará con 1 único programa, que es el siguiente:

`prod_cons_mutex.c` - Código del problema del productor-consumidor resuelto con *mutex* y variables de condición.

COMPILACIÓN - `gcc -o prod_cons_mutex prod_cons_mutex.c -pthread`

EJECUCIÓN - `./prod_cons_mutex`

Es necesario que en el mismo directorio del ejecutable exista un archivo de texto con el nombre `archivo.txt`, el cual contenga números enteros separados por espacios.

FUNCIONAMIENTO - Tras *inicializar* las *variables* pertinentes, poniendo los elementos del *buffer*, las sumas, el *mutex* y 2 variables de condición a 0, se *crean dos hilos*, uno para el productor y otro para el consumidor, y el *thread main* esperará por ambos mediante un *join*.

En el **productor** se hace lo siguiente,

1. Se *abre* el archivo `archivo.txt` en modo de lectura.
2. Se entra a un *bucle* de 80 iteraciones, que también puede terminar en caso de que se llegue al final del archivo leído.
3. Se accede a la región crítica, con lo que se *bloquea el mutex* para evitar el acceso al *buffer* por parte del consumidor. Esto se logra mediante la función `pthread_mutex_lock`.

4. Mientras la cola esta llena, lo que se comprueba recorriéndola hasta encontrar una posición con el valor 0, se **bloquea la variable de condición del propio productor**, para evitar el avance de este.
5. Se ejecuta **produce_item**, con lo que se lee el siguiente entero del archivo y se añade a la suma del productor. En caso de obtener un EOF, se considera el entero 101 y no se suma.
6. Se ejecuta **insert_item**, con lo que se busca la siguiente posición vacía del *buffer* para escribir ahí el entero leído y se actualiza **siguiente_posicion** (esta variable se emplea para dotar al *buffer* del comportamiento FIFO pedido).
7. Se **libera la variable de condición del consumidor**, para indicarle que existe un entero en el *buffer* que puede retirar.
8. Se **libera el mutex** para indicar que el productor ha abandonado la región crítica.
9. Se **duerme** una cierta cantidad de segundos, dependiendo de la iteración del bucle.
10. Fuera del bucle, en caso de completar las 80 iteraciones y no alcanzar el EOF del archivo, se **inserta un código 101 en el buffer**, lo que se logra repitiendo los pasos del 3 al 8.
11. Finalmente, se **cierra archivo.txt**.

Simultáneamente, en el **consumidor** se hace lo siguiente:

1. Se entra a un **bucle** de 80 iteraciones, que también puede terminar en caso de que se haya leído un código 101 de EOF y el *buffer* esté vaciado por completo.
2. Se accede a la región crítica, con lo que se **bloquea el mutex** para evitar el acceso al *buffer* por parte del productor.
3. Mientras la cola está vacía, lo que se logra leyéndola hasta encontrar un valor distinto de 0, se **bloquea la variable de condición del propio consumidor**, para evitar su avance.
4. Se ejecuta **remove_item**, con lo que se retira el elemento del *buffer* más antiguo en ser insertado (gracias al entero auxiliar **siguiente_posicion**) y se sustituye por un 0. También se comprueba si se ha leído un 101 y si se ha vaciado la cola.
5. Se ejecuta **consume_item**, con lo que se añade el entero leído a la suma del consumidor.
6. Se **libera la variable de condición del productor**, para indicarle que existe un hueco en la cola en el que insertar un nuevo *item*.
7. Se **libera el mutex** para indicar que el consumidor ha abandonado la región crítica.
8. Se **duerme** una cierta cantidad de segundos, dependiendo de la iteración del bucle.

Finalmente, desde el hilo **main**, se **imprimen las sumas** de enteros calculada por el productor y por el consumidor, y se muestra la **diferencia entre ambas**. También se **destruyen el mutex y las dos variables de condición**.

CONCLUSIONES

Al ejecutar el programa con un archivo de ejemplo, con el contenido "58 19 61 36 26 52 68 45 21 5 96 52 27 15 18 3 2 1 54 58 19 61 36 26 52 68 45 21 5 96 52 27 15 18 3 2 1 54 58 19 61 36 26 52 68 45 21 5 96 52 27 15 18 3 2 1 54 58 19 61 36 26 52 68 45 21 5 96 52 27 15 18 3 2 1 54 7 85 67 24 58 68 95 55 16", se obtienen los siguientes resultados:

- El productor y el consumidor llegan a la **misma suma** (2819). Esto concuerda con el hecho de que la implementación del productor-consumidor empleando *mutex* y variables de condición **elimina las condiciones de carrera**, al restringir los accesos a la región crítica por más de un hilo.
- En las primeras 30 iteraciones, el productor no duerme y avanza más rápido que el consumidor, con lo que la **cola** está completamente **llena** y el **productor** acaba siendo **ralentizado por el consumidor**, adaptando su velocidad a este último.

Dada la diferencia entre velocidades del productor y el consumidor, en cierto punto del proceso ambos hilos duermen 2 segundos (pues el productor pasa de la iteración 30 mientras el consumidor se encuentra antes de esta). Aun así, el comportamiento del código sigue siendo el mismo que antes.

```
[PROD] Iteración 30 alcanzada. Productor más lento ahora.  
[CONS] Consumido el item 61  
[PROD] Producido el item 27  
[CONS] Consumido el item 36  
[PROD] Producido el item 15  
[CONS] Consumido el item 26  
[PROD] Producido el item 18  
[CONS] Consumido el item 52  
[PROD] Producido el item 3  
[CONS] Consumido el item 68  
[PROD] Producido el item 2  
[CONS] Consumido el item 45  
[PROD] Producido el item 1  
[CONS] Consumido el item 21  
[PROD] Producido el item 54  
[CONS] Consumido el item 5  
[PROD] Producido el item 58  
[CONS] Consumido el item 96  
[PROD] Producido el item 19  
[CONS] Consumido el item 52  
[CONS] Iteración 30 alcanzada. Consumidor más rápido ahora.
```

Diferencia entre el productor y el consumidor para llegar a la iteración 30

- En las iteraciones entre la 30 y la 60, el consumidor no duerme y avanza más rápido que el productor, con lo que el **buffer** se **vacía** casi de inmediato, alternando los roles previos y siendo ahora el **consumidor** el que debe esperar por el productor.

En este caso, el consumidor retira de la cola los enteros recién insertados por el productor en el *buffer*, de modo que este solo se encuentra vacío o con 1 solo *item* todo el tiempo.

```
[CONS] Iteración 30 alcanzada. Consumidor más rápido ahora.  
[CONS] Consumido el item 27  
[CONS] Consumido el item 15  
[CONS] Consumido el item 18  
[CONS] Consumido el item 3  
[CONS] Consumido el item 2  
[CONS] Consumido el item 1  
[CONS] Consumido el item 54  
[CONS] Consumido el item 58  
[PROD] Producido el item 61  
[CONS] Consumido el item 19  
[CONS] Consumido el item 61  
[PROD] Producido el item 36  
[CONS] Consumido el item 36
```

Consumidor vaciando el buffer rápidamente.

- En las iteraciones a partir de la 60, ambos hilos duermen aleatoriamente entre 0 y 3 segundos, pero, por el comportamiento previo del proceso, la cola sigue mayoritariamente vacía. En promedio, la aleatoriedad de los sleeps hace que el buffer se mantenga entre 0 y 2 o 3 items.